

NOTE:

Performance limitations due to low availability of threads on execution machine(2 threads only). Parallel is supposed to be faster when provided with higher no of inputs and higher no of threads. Suggested to run on high thread machine for performance analysis with high no of inputs.

CODE:

```
#include <iostream>
#include <vector>
#include <omp.h>
#include <iomanip>

using namespace std;

// Utility function to print array
void printArray(const vector<int> &arr) {
    for (int x : arr) cout << x << " ";
    cout << endl;
}

// ----- BUBBLE SORT -----

// Sequential Bubble Sort
void bubbleSortSeq(vector<int> &arr) {
    int n = arr.size();
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1])
                swap(arr[j], arr[j+1]);
        }
    }
}

// Parallel Bubble Sort (Odd-Even Transposition)
void bubbleSortParallel(vector<int> &arr) {
    int n = arr.size();

    for (int i = 0; i < n; i++) {

        // Even phase
```

```

        #pragma omp parallel for
        for (int j = 0; j < n-1; j += 2) {
            if (arr[j] > arr[j+1])
                swap(arr[j], arr[j+1]);
        }

        // Odd phase
        #pragma omp parallel for
        for (int j = 1; j < n-1; j += 2) {
            if (arr[j] > arr[j+1])
                swap(arr[j], arr[j+1]);
        }
    }
}

// ----- MERGE SORT -----

void merge(vector<int> &arr, int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++) L[i] = arr[l+i];
    for (int i = 0; i < n2; i++) R[i] = arr[m+1+i];

    int i = 0, j = 0, k = l;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

// Sequential Merge Sort
void mergeSortSeq(vector<int> &arr, int l, int r) {
    if (l >= r) return;

    int m = (l + r) / 2;
    mergeSortSeq(arr, l, m);
    mergeSortSeq(arr, m+1, r);
}

```

```

        merge(arr, l, m, r);
    }

// Parallel Merge Sort
void mergeSortParallel(vector<int> &arr, int l, int r) {
    if (l >= r) return;

    int m = (l + r) / 2;

    #pragma omp task shared(arr)
    mergeSortParallel(arr, l, m);

    #pragma omp task shared(arr)
    mergeSortParallel(arr, m+1, r);

    #pragma omp taskwait
    merge(arr, l, m, r);
}

// ----- MAIN -----

int main() {
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    vector<int> arr(n);

    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    // Copies for each algorithm
    vector<int> a1 = arr, a2 = arr, a3 = arr, a4 = arr;

    double t1, t2, t3, t4;
    double start, end;

    cout << fixed << setprecision(6); // format timing

    // ---- Bubble Sort ----
    start = omp_get_wtime();
    bubbleSortSeq(a1);

```

```

    end = omp_get_wtime();
    t1 = end - start;

    start = omp_get_wtime();
    bubbleSortParallel(a2);
    end = omp_get_wtime();
    t2 = end - start;

    // ---- Merge Sort ----
    start = omp_get_wtime();
    mergeSortSeq(a3, 0, n-1);
    end = omp_get_wtime();
    t3 = end - start;

    start = omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp single
        mergeSortParallel(a4, 0, n-1);
    }
    end = omp_get_wtime();
    t4 = end - start;

    // ---- Output ----
    cout << "\n\n--- Execution Time Comparison ---\n";
    cout << "Sequential Bubble Sort : " << t1 << " sec\n";
    cout << "Parallel Bubble Sort : " << t2 << " sec\n";
    cout << "Sequential Merge Sort : " << t3 << " sec\n";
    cout << "Parallel Merge Sort : " << t4 << " sec\n";

    cout << "\nSorted Array (Parallel Bubble): ";
    printArray(a2);

    cout << "\nSorted Array (Parallel Merge) : ";
    printArray(a4);

    return 0;
}

```

OUTPUT:

```
[dbmslab@fedora BE HPC]$ g++ -fopenmp HPC_2.cpp -o HPC2
^[[A[dbmslab@fedora BE HP./HPC2
Enter number of elements: 9
Enter elements:
23
45
456
1
34
56

23
34
45

--- Execution Time Comparison ---
Sequential Bubble Sort : 0.000002 sec
Parallel   Bubble Sort : 0.000181 sec
Sequential Merge Sort  : 0.000007 sec
Parallel   Merge Sort  : 0.000057 sec

Sorted Array (Parallel Bubble): 1 23 23 34 34 45 45 56 456
Sorted Array (Parallel Merge) : 1 23 23 34 34 45 45 56 456
```

```
dbmslab@fedora BE HPC]$ g++ -fopenmp HPC_2.cpp -o HPC2
^[[A[dbmslab@fedora BE HP./HPC2
Enter number of elements: 9
Enter elements:
23
45
456
1
34
56

23
34
45
```

--- Execution Time Comparison ---

Sequential Bubble Sort : 0.000002 sec

Parallel Bubble Sort : 0.000181 sec

Sequential Merge Sort : 0.000007 sec

Parallel Merge Sort : 0.000057 sec

Sorted Array (Parallel Bubble): 1 23 23 34 34 45 45 56 456

Sorted Array (Parallel Merge) : 1 23 23 34 34 45 45 56 456

OUTPUT with 8 core environment and 500000 numbers:

--- Execution Time Comparison (n = 500000) ---

Sequential Bubble Sort : 1936.307000 sec

Parallel Bubble Sort : 524.157000 sec

Sequential Merge Sort : 0.283000 sec

Parallel Merge Sort : 0.852000 sec

Time required in creating and handling of multiple threads is insignificant for such large inputs.